

Module 3 — Pythia8 Processes & Analysis

Supplementary to the sPHENIX Standalone Course

1. Process groups at a glance

Pythia groups processes into families. Turning on a family enables dozens of 2→2 or 2→3 subprocesses at once.

	Subprocesses	Typical use
l	Elastic, diffractive, non-diff	Minimum-bias
onDiffractive	Non-diff MB only	UE, underlying event studies
ll	qq→qq, qg→qg, gg→gg, ...	Jet samples
g2gg	Only gg→gg	Gluon-rich jet samples
m:all	All J/ψ, χ _c production	Open and hidden charm
m:all	All Y(nS), χ _b production	Open and hidden bottom
ton:all	qg→qγ, qq→gγ, ...	Direct-photon analyses
eBoson:all	W and Z production	Electroweak boson
	tt $\bar{t}$, single-top	Top physics

2. Kinematic cuts

Without a hard cut, soft QCD dominates and you'll fill your disk with useless events. Always set PhaseSpace:pTHatMin when using HardQCD:

```
HardQCD:all = on
PhaseSpace:pTHatMin = 20.    // GeV
PhaseSpace:pTHatMax = -1.    // -1 = no upper cut
```

Note: For mixed p_T samples, run several p_T -hat bins separately and weight each by its cross section (accessed with `pythia.info.sigmaGen()`). This is how ATLAS and CMS produce their JZ1–JZ7 samples.

3. Random seeds and reproducibility

Set `Random:setSeed = on` and `Random:seed = N` where `N` is 0 (time) or 1..900000000 (deterministic). For a production, use a different deterministic seed per job.

```
Random:setSeed = on
Random:seed = 42
```

4. Writing HepMC2 output

```
#include "Pythia8Plugins/HepMC2.h"
HepMC::Pythia8ToHepMC ToHepMC;
HepMC::IO_GenEvent hepmcout("events.hepmc", std::ios::out);
while (numEv < N) {
    if (!pythia.next()) continue;
    HepMC::GenEvent* evt = new HepMC::GenEvent();
    ToHepMC.fill_next_event(pythia, evt);
    hepmcout << evt;
    delete evt;
    ++numEv;
}
```

5. Parsing HepMC2 back

```
#include "HepMC/IO_GenEvent.h"
#include "HepMC/GenEvent.h"
HepMC::IO_GenEvent input("events.hepmc", std::ios::in);
HepMC::GenEvent* evt = nullptr;
while ((evt = input.read_next_event())) {
    for (auto p = evt->particles_begin(); p != evt->particles_end(); ++p) {
        auto part = *p;
        if (part->status() != 1) continue;    // stable only
        double px = part->momentum().px();
        ...
    }
    delete evt;
}
```

6. HepMC2 vs HepMC3

	HepMC2	HepMC3
	2006	2019
	Legacy, GenEvent-centric	Modernized, container-based
	Still ubiquitous in RHIC tools	Adopted by LHC Run 3
on	ASCII, can gzip externally	Native <code>.hepmc3</code> and ROOT
	Fixed set	User-extensible key/value

sPHENIX uses HepMC2 because HIJING's writer was originally HepMC2. Unless you're interfacing with a pure LHC Run 3 workflow, HepMC2 is fine.

7. Charged multiplicity example

```
TH1F h("n_ch", "charged mult", 80, 0, 80);
while (pythia.next()) {
    int n = 0;
    for (int i=0; i<pythia.event.size(); ++i) {
        auto& p = pythia.event[i];
        if (p.isFinal() && p.isCharged() && std::abs(p.eta())<0.5) ++n;
    }
    h.Fill(n);
}
```

Expected: for 200 GeV pp with SoftQCD:nonDiffractive, mean $|\eta|<0.5$ charged multiplicity is about 2.4 (PHENIX/STAR published values). If you see ~ 1.0 you selected elastic+diffractive; if you see ~ 10 you're using heavy-ion defaults.

8. p_T spectrum fit

```
TH1F pt("pt", "charged pT", 100, 0, 10);
// ... fill ...
pt.Fit("expo", "", "", 0.5, 3.0);
```

9. Reweighting hard-process samples

Each job has sigmaGen and nAccept. For a combined histogram across pT-hat slices:

```
double w = pythia.info.sigmaGen() / pythia.info.nAccepted();  
h.Fill(pt, w);
```